ML-Driven PVT-Variation-Aware Leakage Estimation

Digital VLSI Design, Course Project Report
Team 22 (Madhurbala_riders): 45nm_MGK, $V_{DD} = 0.9\,\text{V}$, 8-bit parity checker

Aryan Agrawal (2023102050) Manikya Pant (2023112024)
Vansh Jaiswal (2024102013) Saarthak Sabharwal (2023102055)

Repository: https://github.com/VaNsH-IIIT/dvd-ml-leakage-estimation

1 Introduction

Leakage current sets the static power of a CMOS circuit, and it changes strongly with the supply voltage, the temperature and the process. Finding it with SPICE for every corner is slow, so the project asks for machine-learning models that predict it instead. We built them in three steps:

- **Part A, transistors:** ML models of the four terminal currents (I_d, I_g, I_s, I_b) of one transistor in four cases (NMOS ON, NMOS OFF, PMOS ON, PMOS OFF), trained on SPICE data with random V_{DD}, temperature, width, terminal voltages and process parameters;
- **Part B, gates:** the leakage of INV, NAND2, NOR2, AND2 and OR2 for every input combination, computed only from those transistor models, and checked against SPICE of the whole gate;
- **Part C, test circuit:** the leakage of our 8-bit parity checker as the sum of its gate leakages, checked against SPICE of the full 112-transistor circuit.

Figure 1 shows the flow. Our final results are a **0.51 % mean error (99.49 % mean accuracy)** for both the gates and the parity checker, and 0.12–0.23 % median test error for the transistor models.

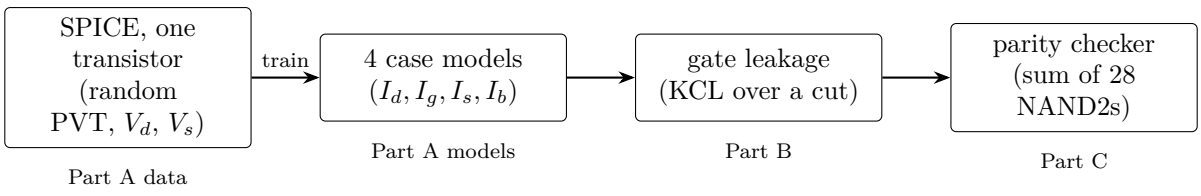


Figure 1: Overall flow. Each stage is checked against SPICE of the same transistor, gate or circuit.

We used ngspice-42 for all simulations and Python 3.12 with scikit-learn 1.9.1, LightGBM 4.7.0, NumPy 2.5.4, SciPy 1.18.1 and pandas 3.0.6 for the models. All results regenerate from the scripts in `scripts/` with fixed seeds; the commands, in order, are in the `README.md` of our repository.

2 Setup and Choices

Table 1: Our assignment and simulation setup.

Item	Value
Model file	`45nm_MGK.pm` (PTM, BSIM4 level 54), from the course `model.tar`
Nominal V_{DD}	0.9 V (sampled 0.81–0.99 V)
Channel length, widths	$L = 45$ nm; $W = 1$–$8 \times L_{min} = 45$–360 nm
Test circuit	8-bit parity checker (7 XOR2 = 28 NAND2, 112 transistors)
Simulator	ngspice-42, batch mode, DC operating point (`op`)
Simulator options	`.options gmin=1e-18 abstol=1e-18 reltol=1e-6 vntol=1e-9`

Before generating any data we checked our simulation driver against the professor's sample netlist `22n_NMOS_ON.net`: it reproduces all 48 points to within 4×10^{-5} relative difference (`scripts/verify_sim.py`).

Confirmed with the TAs.

- **Widths:** L is fixed at 45 nm and the widths vary between 1 and $8 \times L_{min}$, also in the gates and the test circuit.
- **XOR2** for the parity checker is the standard four-NAND2 circuit, since XOR is not one of the five gates.
- **Accuracy target:** "up to 99 % mean accuracy" means a mean per-sample error below 1 %.
- **Training-set size:** training on more than the 10,000 samples per case is allowed. The project's 10,000-row datasets still provide all validation and test rows, and a data-size study shows what the 7,000 training rows alone give.
- **Model comparison:** at least four techniques are compared for each transistor case, and one final model is kept per case.

Our assumptions (where the document and the TAs' answers leave room):

- **Leakage of a gate or circuit** is all the current drawn from V_{DD}-level sources: the V_{DD} supply current plus the current drawn by the inputs held at V_{DD}, i.e. the static power divided by V_{DD}. The TAs left it to us to work out which currents count. Section 5 shows what this leakage consists of (gate tunnelling vs sub-threshold and junction current), and Section 6 shows why the narrower "V_{DD} pin only" definition does not add up to the circuit leakage. We report the total leakage; we did not separate the junction current from the sub-threshold current.
- **Widths in a gate or circuit:** the document says all NMOS in a gate share one draw and all PMOS share another. We apply this to the widths too: each corner draws one NMOS width W_n and one PMOS width W_p (uniform in 45–360 nm), shared by every NMOS and every PMOS of the gate or circuit at that corner.
- **Where the accuracy target applies:** we apply the 1 % mean-error target to the leakage estimates of the gates and the circuit, which are the project's outputs. For the transistor models we report the median, p95 and mean per current; Section 4 explains why the mean alone is misleading for currents that cross zero.

Metrics. Throughout the report, a sample's error is $|ML - SPICE|/|SPICE|$. We summarise it by its median, its 95th percentile (p95: 95 % of the samples are at or below it) and its mean; "mean accuracy" = 100 %− mean error.

3 Part A: Data Generation

Sampling (`sim/spec.py`, `scripts/a1_generate_data.py`). For each case we draw 10,000 samples independently and uniformly, with a fixed seed per case: V_{DD} in 0.81–0.99 V (drawn first), V_d and V_s each in 0–V_{DD}, W in 45–360 nm, T in −40 to 125 °C, and `vth0`, `toxe`, `ndep`, `u0` and `xj` each at nominal ±10 % of the 45nm_MGK card value (NMOS and PMOS drawn separately). The gate and body voltages (V_g, V_b) are fixed by the case: NMOS ON $(V_{DD}, 0)$, NMOS OFF $(0, 0)$, PMOS ON $(0, V_{DD})$, PMOS OFF (V_{DD}, V_{DD}).

Netlists (`sim/runner.py`). Each sample is one netlist with the model card written inline with the sample's process values, one MOSFET with $L = 45$ nm and the sampled W, one voltage source per terminal, the project's `.options` line, and the sampled temperature. The terminal currents are ngspice's `i(Vd)`, `i(Vg)`, `i(Vs)` and `i(Vb)`, positive when current flows out of the device terminal. The four datasets, `data/partA/<CASE>.csv`, have exactly the project's columns. The 40,000 samples take about 20 s to simulate on 16 cores.

Split. Each dataset is split 70/15/15 % (7,000 / 1,500 / 1,500 rows) with seed 22. For the final models the training part is enlarged with up to 80,000 extra samples per case, drawn the same way with other seeds (`data/partA_extra/`); validation and test always stay the project's rows. The test rows are never used to choose or tune a model.

Data check (`scripts/a2_check_data.py`). Every row has all values, inside its range. Kirchhoff's law holds in every row: the four currents balance to within 10^{-15} A or 0.1 % of the largest current (median residual about 7×10^{-7} of the largest current, from ngspice's 6–7 printed digits). The trends are physical (Figure 2): the OFF current rises ×115 (NMOS) and ×12 (PMOS) from −40 to 125 °C, falls as |`vth0`| rises (×0.14 and ×0.33 over ±10 %), and the gate current falls ×0.07 as `toxe` rises by 20 %.

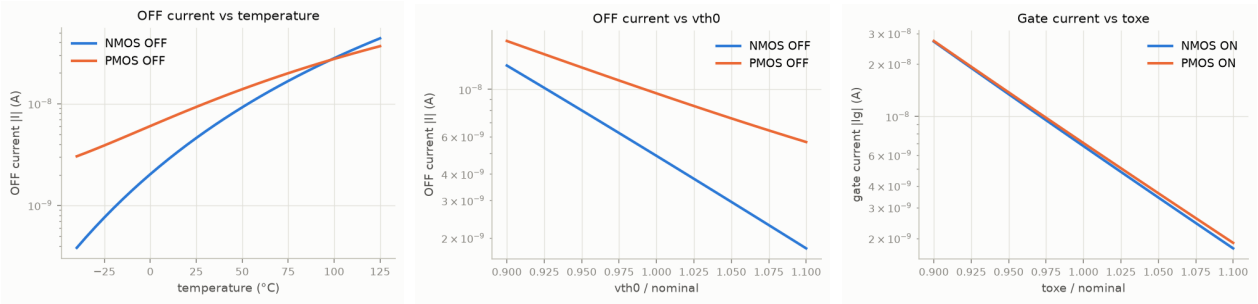


Figure 2: Trend checks, one parameter swept from the nominal point ($W = L_{min}$, $V_{DD} = 0.9$ V, 27 °C). OFF current at $V_{ds} = V_{DD}$; gate current of ON devices at $V_d = V_s$.

Structure in the data that shaped the models.

1. The device is drain/source symmetric: swapping V_d and V_s swaps I_d and I_s. If we rename the terminals "hi" (higher voltage) and "lo", three of the four currents of every case keep one sign over the whole dataset.
2. The exception is one terminal per OFF case (lo for NMOS, hi for PMOS), where the outgoing channel current and the incoming gate-tunnelling current can cancel. In 8.0 % (NMOS) and 4.8 % (PMOS) of the OFF rows it is below 3 % of the gate current.
3. The OFF body current is about 10^{-14} A, nearly constant, and 6–8 decades below the gate leakages (8.8×10^{-9} to 1.7×10^{-6} A).

4 Part A: Transistor Models

4.1 Preprocessing

Each sample is rewritten in the hi/lo form above, and predictions are mapped back to I_d and I_s. The model has 17 inputs: T and $1/T$, V_{DD}, W/L and $\log(W/L)$, V_{hi}, V_{lo}, $V_{hi} - V_{lo}$ and its $\log_{10}$, $V_g - V_{hi}$, $V_g - V_{lo}$, $V_b - V_{lo}$, and the five process deviations, all standardised. The currents span 7–10 decades, so each target is $\operatorname{asinh}(I/I_0)$, which behaves like log-current: a fixed error in the target is a fixed relative error in the current, and it is smooth through zero.

- For the fixed-sign currents, $I_0 = 10^{-3}\times$ the median $|I|$ of the training set.
- For the mixed-sign OFF terminal, $I_0 = 0.1\,|I_g|$ of the same sample (at prediction time, the model's own I_g prediction), so its zero crossing becomes a gentle slope. This cut that terminal's validation error about threefold (Section 7).

One regressor is trained per current. A saved case model (`trained_models/<CASE>.joblib`) maps the project's input columns to (I_d, I_g, I_s, I_b).

4.2 Comparison of four techniques

We compared four model types on each case (`ml/models.py`, `scripts/a3_compare_models.py`):

1. **Polynomial:** degree-3 polynomial features + ridge regression ($\alpha = 10^{-4}$);
2. **LightGBM:** 4,000 trees, learning rate 0.03, 63 leaves, at least 10 rows per leaf, row and column subsampling 0.8;
3. **MLP:** 3 hidden layers × 128 tanh units, Adam with learning rate 2×10^{-3}, L2 penalty 10^{-6}, early stopping on 10 % of the training rows (patience 200 epochs);
4. **Gaussian process (GP):** ARD RBF kernel (one length scale per input) × constant + white noise; kernel fitted on 1,000 training rows, then conditioned on all of them.

Table 2: Model comparison on the validation set, 7,000 training rows: R^2 of the worst of the four currents (on the asinh target), and median / p95 error pooled over the four currents. Bold: lowest p95 per case.

Case	Metric	Polynomial	LightGBM	MLP	Gaussian process
NMOS ON	R^2	0.9840	0.9979	0.9995	0.9989
	median / p95	8.72 / 56.7 %	3.16 / 26.8 %	**0.43 / 2.58 %**	0.11 / 3.99 %
NMOS OFF	R^2	0.920	0.923	0.898	0.902
	median / p95	7.98 / 84.5 %	2.31 / 31.2 %	**1.06 / 10.5 %**	0.53 / 15.8 %
PMOS ON	R^2	0.9891	0.9982	0.9999	0.9986
	median / p95	5.51 / 29.5 %	2.53 / 19.2 %	0.41 / 2.13 %	**0.07 / 1.47 %**
PMOS OFF	R^2	0.875	0.891	0.853	0.905
	median / p95	8.81 / 86.3 %	2.26 / 29.9 %	**1.04 / 8.45 %**	0.29 / 10.7 %

The polynomial is too stiff for 17 inputs and many decades of current, and the trees predict in steps, which does not suit a smooth function that has to be within 1 %. The GP has the best median in every case, but a heavy tail: one smooth kernel cannot follow the steep regions, and it falls back towards the mean at the edges of the input box. The MLP has the lowest p95 in three of the four cases. The lower R^2 in the OFF cases comes from the body current, which is $\sim 10^{-14}$ A and nearly constant, so its R^2 says little; the other three OFF currents have $R^2 \geq 0.9996$ for the MLP.

Choice rule: lowest validation p95, then lowest median. We choose on the tail rather than the median because the gates put transistors at the edges of the training box; the GP shows why (it gives 12.6 % median error at gate level, Section 8). No gate or circuit result is used to train or choose a model.

4.3 More data and ensembles

Data size. We retrained the MLP on less and more data. Validation and test stay on the project's dataset.

Table 3: MLP validation median / p95 error vs training rows.

Case	2,000	4,000	7,000	27,000	47,000
NMOS ON	0.75 / 7.56 %	0.67 / 4.49 %	0.43 / 2.58 %	0.31 / 1.38 %	0.32 / 1.47 %
NMOS OFF	1.44 / 16.7 %	0.81 / 10.4 %	1.06 / 10.5 %	0.71 / 6.96 %	0.71 / 7.66 %
PMOS ON	1.00 / 7.60 %	0.46 / 2.51 %	0.41 / 2.13 %	0.34 / 1.42 %	0.32 / 1.29 %
PMOS OFF	1.19 / 14.9 %	0.68 / 10.1 %	1.04 / 8.45 %	0.58 / 8.14 %	0.64 / 7.36 %

At 2,000 rows every case is clearly worse. Above 7,000 rows the tail improves up to 27,000 rows, after which a single network levels off: its remaining error is mostly run-to-run variance. More data costs little simulation time (about 20 s for 40,000 samples) but longer training (1–2.4 h per case for one 87,000-row ensemble).

Ensembles. Averaging three MLPs trained with different seeds removes most of that variance. We trained three ensemble variants on 87,000 rows, which differ in what the networks learn:

- **per-width targets:** current / (W/L), since leakage scales almost linearly with width;
- **plain-current targets:** the current itself;
- **linear baseline:** a ridge fit first, with the networks learning only what it leaves. Log-current is close to linear in most inputs, and the linear part carries those trends into the corners of the input box, where training rows are sparse.

Their errors are partly independent, so we also scored equal-weight averages of their predicted currents (Table 4).

Table 4: Validation median / p95 error, 87,000 training rows. Bold: chosen per case.

Candidate	NMOS ON	NMOS OFF	PMOS ON	PMOS OFF
Ensemble, per-width targets	0.18 / 0.87 %	0.44 / 4.96 %	0.18 / 0.71 %	0.38 / 4.59 %
Ensemble, plain-current targets	0.20 / 0.92 %	0.30 / 4.02 %	0.20 / 0.80 %	0.40 / 4.49 %
Ensemble, linear baseline	0.18 / 0.91 %	0.31 / 3.52 %	0.18 / 0.87 %	0.29 / 4.44 %
Average, plain-current + linear	0.15 / 0.68 %	**0.24 / 3.05 %**	0.14 / 0.62 %	0.25 / 3.48 %
Average of all three	**0.13 / 0.60 %**	0.23 / 3.22 %	**0.12 / 0.52 %**	**0.22 / 3.40 %**

So the final model of each case is an MLP ensemble: the average of all three variants for NMOS ON, PMOS ON and PMOS OFF, and of the plain-current and linear-baseline variants for NMOS OFF.

4.4 Final models on the test set

Table 5: Chosen models on the 1,500 test rows per case, errors pooled over the four currents, and median / p95 error per current. *Mean over the currents with $|I| \geq 10^{-11}$ A only.

Case	Median	p95	Mean	Mean*	I_d	I_g	I_b
NMOS ON	0.13 %	0.61 %	24.1 %	3.03 %	0.12 / 0.7	0.12 / 0.7	0.14 / 0.5
NMOS OFF	0.23 %	3.16 %	2.15 %	0.36 %	0.22 / 1.3	0.16 / 0.5	0.51 / 22.0
PMOS ON	0.12 %	0.53 %	0.40 %	0.43 %	0.13 / 0.5	0.11 / 0.6	0.13 / 0.5
PMOS OFF	0.22 %	3.05 %	3.28 %	0.29 %	0.19 / 1.0	0.14 / 0.5	0.82 / 20.0

- Test and validation agree to within 0.01 % in the median, so there is no sign of over-fitting to the validation split. (I_s behaves like I_d; all numbers are in `reports/partA/MODELS.md`.)
- In the OFF cases the p95 comes almost entirely from the $\sim 10^{-14}$ A body current. The three currents that matter have p95 ≤ 1.4 %.
- The means are pulled up by a few near-zero currents. In NMOS ON, when $V_d \approx V_s$ the channel current crosses zero ($|I_d|$ down to 1.6×10^{-12} A against $\sim 10^{-4}$ A at full bias); one such sample alone is 78,000 % off, with an absolute error of only 10^{-9} A. These channel currents never enter the gate leakage (Section 5).

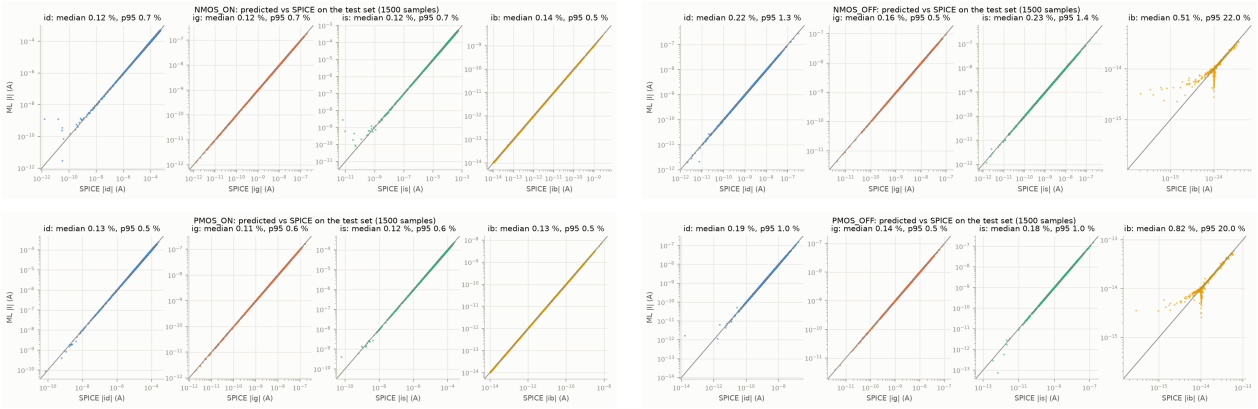


Figure 3: Chosen models on the test set: predicted vs SPICE $|I|$ for each terminal current.

5 Part B: Gate Leakage

Gates (`sim/gates.py`, `scripts/b1_gates_spice.py`). We used standard static CMOS gates, with AND2/OR2 = NAND2/NOR2 + INV. Each gate was simulated for every input combination at 200 corners, each with its own V_{DD}, T, NMOS and PMOS process draws, and widths W_n and W_p: 3,600 simulations in total. The inputs are at exactly 0 or V_{DD}.

What is the leakage made of? We repeated every gate simulation with the model's gate tunnelling switched off (`igcmod = igbmod = 0`, `scripts/b3_leakage_components.py`). What remains is sub-threshold plus junction current.

Table 6: Make-up of the gate leakage (medians over the 200 corners).

Gate, inputs	SPICE leakage (A)	Gate tunnelling	Sub-threshold + junction	Drawn through inputs at V_{DD}
INV 0 / 1	1.10e-7 / 1.13e-7	65 / 46 %	35 / 54 %	0 / 46 %
NAND2 00 / 11	1.01e-7 / 2.26e-7	97 / 46 %	3 / 54 %	0 / 46 %
NOR2 00 / 11	2.19e-7 / 8.97e-8	64 / 92 %	36 / 8 %	0 / 92 %
All 3,600 runs		67 %	33 %	

With the 0.9 nm oxide of this model card, gate tunnelling is about two thirds of the leakage. Sub-threshold current grows with temperature and takes over when hot: the tunnelling share falls from 89 % below 0 °C to 34 % above 85 °C. When inputs are high, a large part of the leakage (up to 92 % for NOR2 11) is gate tunnelling into the ON NMOS channels, drawn through those inputs. That is why our leakage counts the input currents.

5.1 ML gate leakage

For each gate and corner (`ml/gate_eval.py`, `scripts/b2_gates_ml.py`):

1. each transistor's terminal voltages come from the SPICE node voltages, with exact inputs and rails;
2. its case (ON/OFF) follows from its gate voltage, and that case's model predicts its four currents;
3. the currents are combined into the V_{DD} current by Kirchhoff's law over a **cut through the OFF devices**;
4. the current drawn by each input held at V_{DD} (the gate currents of the transistors it drives) is added.

Why a cut through the OFF devices? Summing the currents at the V_{DD} node needs the channel current of the ON pull-up transistors. These sit at a V_{ds} of microvolts, where their current is a tiny difference set by the OFF network, and the training data has almost no rows there. But Kirchhoff's law holds for any set of nodes S that contains V_{DD} (and not ground or an input): $I_{leak} = -\sum$ of the terminal currents at the nodes in S. For a device with both drain and source inside S, $I_d + I_s = -(I_g + I_b)$, so its channel current cancels exactly. We pick the S whose boundary devices are all OFF, with the largest $|V_{ds}|$. With every transistor simulated alone in SPICE instead of the ML model, this recipe reproduces the full-gate SPICE leakage to within 0.0005 %, so the recipe is exact and the remaining error belongs to the models.

Table 7: How the currents are combined, same models, all 3,600 gate simulations.

Method	Median	p95	Mean
Sum at the V_{DD} node	2.43 %	18.2 %	5.17 %
Minimal supernode (V_{DD} + ON-connected nodes)	0.38 %	1.37 %	0.52 %
Best cut through the OFF devices (used)	**0.37 %**	**1.33 %**	**0.51 %**

5.2 Results

Table 8: Gate leakage, ML vs SPICE, 200 corners per row. Bias = median of ML/SPICE −1.

Gate	Inputs	SPICE median (A)	ML median (A)	Bias	Median error	p95 error
INV	0	1.098e-07	1.097e-07	+0.26 %	0.50 %	1.66 %
INV	1	1.129e-07	1.125e-07	−0.28 %	0.48 %	1.35 %
NAND2	00	1.008e-07	1.016e-07	+0.53 %	0.57 %	1.43 %
NAND2	01	1.579e-07	1.586e-07	+0.10 %	0.34 %	1.18 %
NAND2	10	9.068e-08	9.085e-08	+0.29 %	0.44 %	1.47 %
NAND2	11	2.256e-07	2.250e-07	−0.23 %	0.43 %	1.35 %
NOR2	00	2.189e-07	2.188e-07	+0.27 %	0.48 %	1.54 %
NOR2	01	1.668e-07	1.664e-07	+0.06 %	0.29 %	1.38 %
NOR2	10	9.702e-08	9.693e-08	−0.21 %	0.43 %	1.55 %
NOR2	11	8.966e-08	8.930e-08	−0.33 %	0.48 %	1.43 %
AND2	00	2.420e-07	2.419e-07	+0.13 %	0.30 %	1.10 %
AND2	01	2.823e-07	2.823e-07	−0.17 %	0.30 %	1.16 %
AND2	10	2.128e-07	2.127e-07	−0.12 %	0.27 %	1.18 %
AND2	11	3.466e-07	3.457e-07	−0.12 %	0.28 %	1.01 %
OR2	00	3.478e-07	3.475e-07	−0.21 %	0.38 %	1.27 %
OR2	01	2.875e-07	2.874e-07	+0.12 %	0.32 %	1.05 %
OR2	10	2.230e-07	2.233e-07	+0.00 %	0.28 %	1.01 %
OR2	11	2.299e-07	2.296e-07	−0.02 %	0.28 %	1.33 %
All 3,600 simulations (mean error **0.51 %**, 99.49 % mean accuracy)					**0.37 %**	**1.33 %**

89 % of the gate simulations are within 1 %, and only 0.33 % exceed 5 % (largest 6.9 %).

- **Most accurate: AND2 and OR2** (0.27–0.38 % median). Their leakage is the sum of two stages whose device errors partly offset.
- **Least accurate: the output-high cases NAND2 00 (0.57 %) and INV 0 (0.50 %).** Their leakage flows through OFF NMOS with the drain at V_{DD}, and includes the gate tunnelling of the ON PMOS at $V_d \approx V_s \approx V_{DD}$; both are at edges of the training data.
- **The bias is small** (−0.33 to +0.53 %), so the error is mainly scatter.

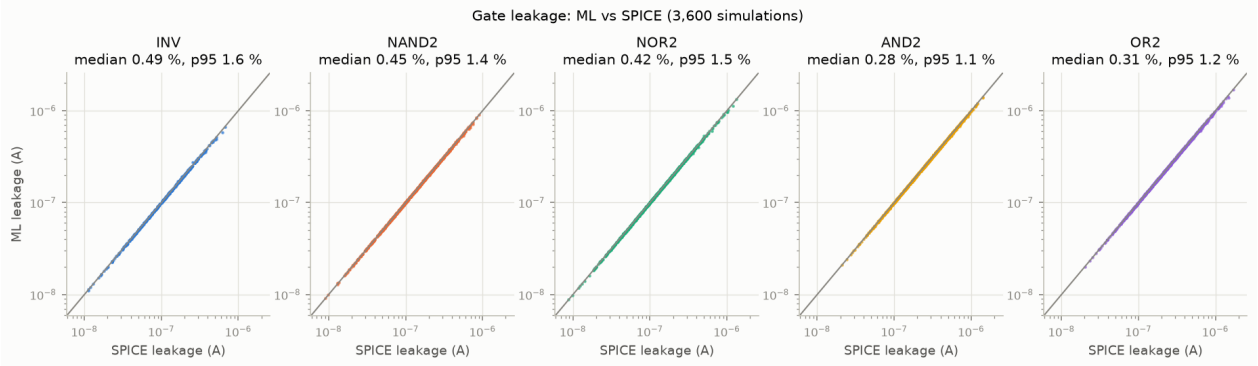


Figure 4: Gate leakage, ML vs SPICE, all 3,600 gate simulations (log axes), one panel per gate.

Internal node voltages. We also solved Kirchhoff's law at each internal node with the ML models. The ML node voltages agree with SPICE to a median of 0.33 mV or better in every gate case, but fail in a few corners where the balance depends on an ON device at microvolt V_{ds} (worst: OR2 inputs 10, node y, 0.77 V).

How the gate results were used. No gate or circuit result was used to train or select a model. The gate runs did shape the method: the cut-based sum, the ensembles and the three ensemble variants were developed after gate runs showed where the earlier models were weak. So, to check the numbers on data that influenced nothing, `scripts/d3_fresh_check.py` draws new transistor samples and 100 new corners with fresh seeds. The gates give 0.37 % median and **0.50 % mean** error there, the same as above.

6 Part C: 8-bit Parity Checker

Circuit (`sim/parity.py`). Seven XOR2 gates in a tree compute the parity of the 8 inputs, and each XOR2 is the standard four-NAND2 circuit (Figure 5): 28 NAND2 and 112 transistors. SPICE simulates the full netlist for all 256 input vectors at the same 200 corners: 51,200 runs, about 4.5 minutes.

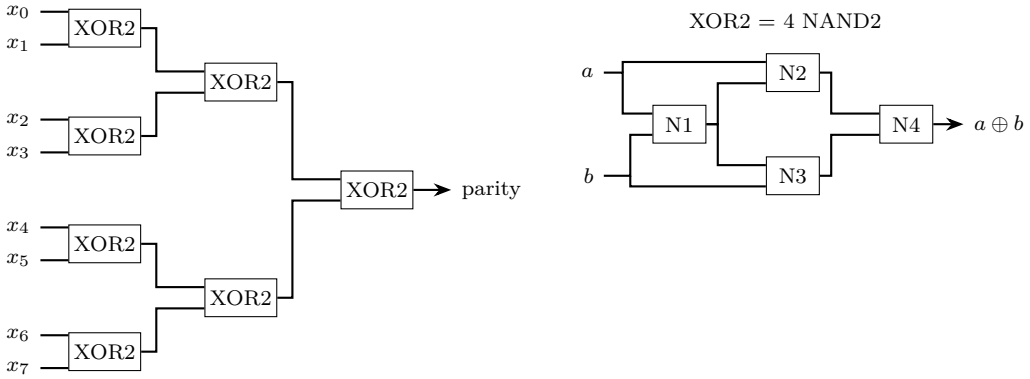


Figure 5: The 8-bit parity checker: a tree of 7 XOR2 gates (left), each built from 4 NAND2 (right).

ML estimate (`scripts/c2_parity_ml.py`). For each input vector we find the input pattern of every NAND2, look up the NAND2 gate-model leakage of Part B at that corner (same widths and process), and add up the 28 values, as the project document prescribes. Nothing else is fitted.

Why the plain sum works. Inside the circuit, the current drawn by an internal input at logic 1 comes from the V_{DD} pin of the gate driving it. A stand-alone gate's V_{DD} current does not include it, but its "drawn through inputs at V_{DD}" part does, so each such current is counted exactly once, in the receiving gate. Adding up stand-alone SPICE NAND2 values and comparing with the full-circuit SPICE isolates this composition step from the ML error (Table 9).

Table 9: Sum of stand-alone SPICE NAND2 leakages vs full-circuit SPICE (51,200 runs).

Leakage definition	Median	p95	Max
V_{DD} supply + inputs at V_{DD} (used)	**0.18 %**	**0.93 %**	**2.15 %**
V_{DD} supply current only	23.9 %	60.7 %	76.0 %

Results (`reports/partC/PARITY.md`). Table 10 gives the same metrics as the gate results table, for all 51,200 runs and by the number of inputs at logic 1.

Table 10: Parity checker leakage, ML (sum of 28 NAND2 gate models) vs full-circuit SPICE.

Vectors	Runs	SPICE median (A)	ML median (A)	Bias	Median err.	p95 err.	Mean err.
all 256	**51,200**	4.761e-06	4.747e-06	−0.22 %	**0.38 %**	**1.35 %**	**0.51 %**
0 inputs at 1	200	4.961e-06	4.952e-06	−0.14 %	0.34 %	1.28 %	0.49 %
2 inputs at 1	5,600	4.799e-06	4.790e-06	−0.20 %	0.35 %	1.35 %	0.50 %
4 inputs at 1	14,000	4.733e-06	4.718e-06	−0.23 %	0.38 %	1.32 %	0.50 %
6 inputs at 1	5,600	4.735e-06	4.732e-06	−0.25 %	0.40 %	1.33 %	0.52 %
8 inputs at 1	200	4.821e-06	4.780e-06	−0.25 %	0.40 %	1.28 %	0.54 %

So the parity checker reaches a **0.51 % mean error (99.49 % mean accuracy)**. 90 % of the runs are within 1 %, and the largest single error is 4.7 %. On the 100 fresh corners (25,600 runs) the mean error is 0.52 %.

- Accuracy is nearly flat across the input vectors (0.34–0.41 % median for 0 to 8 inputs high), because every vector uses the same four NAND2 patterns in different proportions.
- The circuit is as accurate as the gates, not better: at each corner its estimate is a weighted sum of the same four NAND2 gate-model values, so it inherits their error (NAND2 alone: 0.56 % mean), plus the small composition error. With the V_{DD}-pin definition the composition alone would add 24 %.

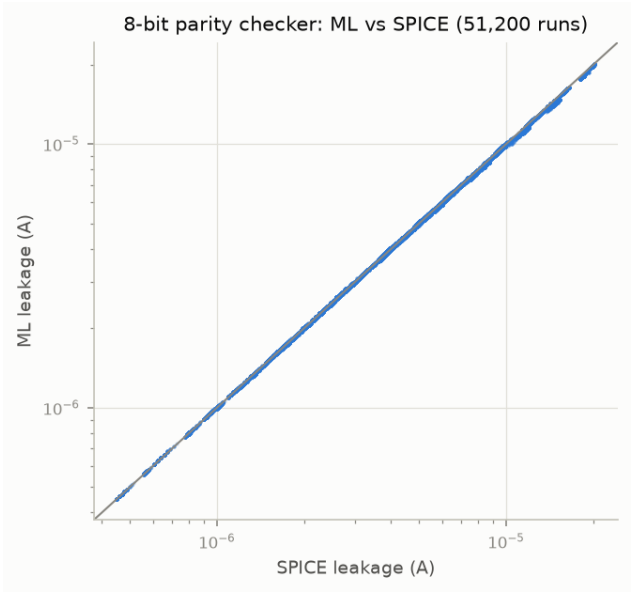


Figure 6: Parity checker, ML vs SPICE leakage for all 51,200 runs (log axes).

7 Error Analysis

Full numbers are in `reports/ERROR_ANALYSIS.md` (`scripts/d1_error_analysis.py`).

Transistor models. The largest relative errors come from four places:

1. **Near-zero currents.** When $|V_d - V_s| < 10\,\text{mV}$ in an ON device, I_d and I_s are tiny differences that can cross zero. The p95 error there is 344 % (NMOS ON), against 0.4 % at $|V_d - V_s| > 0.5\,\text{V}$, but the absolute error is negligible and these currents never enter the gate sums.
2. **The OFF body current** ($\sim 10^{-14}$ A) carries the OFF-case p95 (22 % NMOS, 20 % PMOS). It is 6–8 decades below the gate leakages, so it does not affect Parts B and C.

3. **The mixed-sign OFF terminal** is the hardest real current. Scaling its target by the sample's own gate current cut its validation error from 2.84 to 0.96 % median (NMOS OFF) and from 4.78 to 1.86 % (PMOS OFF) (`scripts/a3c_mixed_scale_study.py`).

4. **Corners of the input box.** The median error is nearly flat across temperature, `vth0` and `toxe`, but the tail grows at the edges, e.g. NMOS ON p95 7.2 % at −40 to −7 °C against 0.7–1.0 % elsewhere.

Gates and circuit. The error grows towards the hot and high-V_{DD} corners, where the leakage is largest or a device sits at the edge of its training range (Figure 7). For the gates, the median / p95 is 0.46 / 2.5 % at 92–125 °C against 0.33–0.38 / 1.0–1.2 % below; for the parity checker 0.75 / 3.4 % against 0.23–0.48 / 0.9–1.2 %. The gate error is also higher at the lowest NMOS `vth0` (−10 to −5 %: 0.50 % median) and for narrow PMOS (45–124 nm: 0.44 / 2.1 % against 0.31 / 1.0 % for the widest). The worst corners combine several extremes: corner 168 (V_{DD} = 0.99 V, NMOS `toxe` −9.6 %, W_n = 306 nm) and corner 173 (124 °C, NMOS `vth0` −8 %) reach 6.9 % and 6.2 % in single gate cases. There the OFF current exceeds anything in the training data and a tanh network flattens out; the linear-baseline variant helps most here.

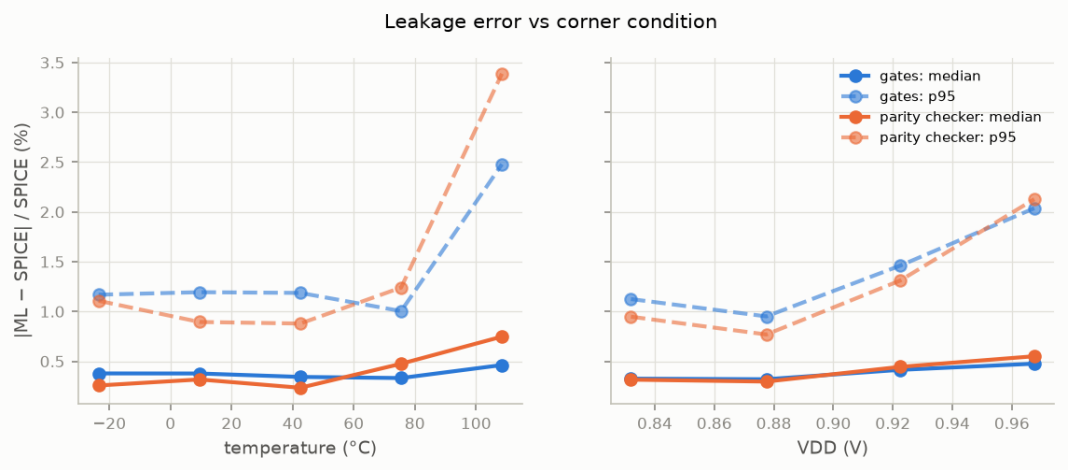


Figure 7: Median and p95 leakage error vs temperature and V_{DD}, for the gates and the parity checker.

8 Limitations and Improvements

- **Edges of the input box.** Gates bias their transistors at the edges of the (V_d, V_s) square, which uniform sampling covers thinly. Adding 10,000 training rows concentrated there made things worse (OFF-case validation median 1.05 → 2.6 %, gate median 1.88 → 2.14 %), so we kept uniform sampling and added more uniform data. A model with physical structure in V_{ds}, e.g. $I \propto 1 - e^{-V_{ds}/v_t}$, would handle the edges better.
- **Joint extremes** (hot + low `vth0`, or thin oxide + high V_{DD}, with a wide device) still reach a few percent in single gate cases. A stress set at these corners in the selection rule, or denser sampling there, would reduce it.
- **Validation accuracy in the bulk is not enough.** The GP has the best validation median (0.07–0.53 %), but in the gates it gives 12.6 % median error: away from its training points an RBF kernel falls back towards the mean, while the MLP extends the trend. This is why we select on the p95.
- **Composition from stand-alone gates.** Inside the circuit the gate inputs are up to 18 mV off the rails, which leaves 0.18 % median between the sum of SPICE gate values and the full circuit.

- **Scope.** One model file, nominal $V_{DD} = 0.9\,\text{V}$, DC leakage only. Within a gate or circuit all NMOS share one width and all PMOS another; mixed sizings need no new models, only new SPICE references.

9 Team Contributions

We split the work equally, one part each, and each part went into the repository as its own commit and pull request.

Member	Work
Aryan Agrawal (2023102050)	Simulation setup: the single-transistor ngspice driver (`sim/spec.py`, `sim/runner.py`) and its check against the professor's sample netlist. Part A data: generating the four 10,000-sample datasets, the data check and trend plots. Section 3.
Vansh Jaiswal (2024102013)	Part A models: features and targets, the comparison of the four techniques, the data-size study, the ensembles and their averages, model selection and the test results (`ml/`, `scripts/a3*`, `scripts/a4*`). Section 4. The GitHub repository and the README.
Saarthak Sabharwal (2023102055)	Part B: the gate netlists and SPICE runs, the ML gate leakage with the cut through the OFF devices, the leakage components and the gate diagnostics (`sim/gates.py`, `ml/gate_eval.py`, `scripts/b*`). Section 5.
Manikya Pant (2023112024)	Part C: the parity checker netlist and SPICE runs, the ML composition, the error analysis and the fresh-data check (`sim/parity.py`, `scripts/c*`, `scripts/d1*`, `scripts/d3*`). Sections 6–8. Putting the report together.

We all wrote parts of the report and checked each other's results. The pull requests were opened from Vansh's GitHub account and reviewed and merged by the owner of each part.

10 Conclusion

Our transistor models, MLP ensembles chosen over a polynomial, LightGBM and a Gaussian process, predict the four terminal currents of the 45nm_MGK device to 0.12–0.23 % median test error. Combined with Kirchhoff's law over a cut through the OFF devices, they give the leakage of the five gates with a **0.51 % mean error (99.49 % mean accuracy)** over 3,600 simulations. With the leakage defined as all current drawn from V_{DD}-level sources, the 8-bit parity checker's leakage is simply the sum of its 28 NAND2 gate leakages, and it reaches the same **0.51 % mean error** over 51,200 runs. On fresh corners that influenced nothing, the gates and the circuit give 0.50 % and 0.52 %. The errors that remain are at the joint extremes of the input range, where the models extrapolate.

Use of AI Tools

An AI assistant was used to support parts of this work. We guided it throughout, reviewed and checked every step and result ourselves, and we take full responsibility for the models, the results and the report, in line with the responsible use of AI.

References

[1] Digital VLSI Design, *ML-Driven PVT-Variation-Aware Leakage Estimation*, project document, IIIT Hyderabad, 2026.

[2] K. Roy, S. Mukhopadhyay and H. Mahmoodi-Meimand, "Leakage current mechanisms and leakage reduction techniques in deep-submicrometer CMOS circuits," *Proc. IEEE*, vol. 91, no. 2, pp. 305–327, 2003.

[3] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson.

[4] BSIM Group, UC Berkeley, *BSIM4 MOSFET Model User's Manual*, https://bsim.berkeley.edu

[5] Predictive Technology Model (PTM), Arizona State University, https://ptm.asu.edu

[6] ngspice, https://ngspice.sourceforge.io

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[8] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," *NeurIPS*, 2017.

[9] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.